Sistema de Reportes con Inteligencia Artificial

Nombre: Valeria Martin Llamas

Carrera: Ingeniería en computación

Materia: Programación para internet

Contenido

Descripción 3

Por qué se creo 4

Objetivo General 4

- Objetivos Específicos 4

Tecnologías utilizadas 5

- Backend 5
- Frontend 6

Arquitectura del proyecto 6

Estructura del proyecto 6

Mapa Interactivo del CUCEI 7

- Descripción del mapa 7

Configuración de la IA (Google Gemini) 8

- ¿Por qué se eligió Gemini? 8
- Modelo usado 8
- Configuración del Cliente 8
- El Prompt 9
  - ¿Por qué este prompt está estructurado así? 10
  - Envío de la imagen a Gemini 10
  - Flujo completo del análisis con la IA 11

Seguridad y Robustez 12

Conclusión 13

Descripción

El sistema de reportes con IA es una página web diseñada para facilitar la gestión de incidencias y problemas de mantenimientos en las instalaciones del CUCEI.

Esta plataforma nos permite a los estudiantes, docentes y personal administrativo reportar cualquier tipo de problema que se detecte dentro de las instalaciones, como:

- Daños estructurales (paredes rotas, goteras, grietas)
- Problemas de limpieza (basura acumulada, áreas sucias)
- Riesgos de seguridad (cables expuestos, pisos mojados, iluminación defectuosa)
- Fallas tecnológicas (computadoras dañadas, proyectores que no funcionan)
- Problemas de servicios (falta de agua, electricidad, mobiliario dañado)

Lo que hace único a este sistema es la integración de la IA usando a Gemini ai de Google, que analiza automáticamente las imágenes subidas por los usuarios para determinar:

Problema: Hace una descripción clara y especifica de lo que ocurre en la imagen

Categoría: Clasificación del tipo de problemas entre las categorías de infraestructura, limpieza, seguridad, tecnología o servicios

Urgencia: Nivel de prioridad entre baja, media o alta basado en el peligro real que detecte la IA.

El sistema no solo permite reportar problemas, sino que también incluye funcionalidades sociales como likes y comentarios para que los usuarios puedan dar sus opiniones o casos con ese problema.

Por qué se creo

Este proyecto nació de la necesidad de **automatizar y agilizar** el proceso de reporte de incidencias en entornos como:

- Instituciones educativas (escuelas, universidades)
- Edificios corporativos
- Centros comerciales
- Espacios públicos

Beneficios:

- ✅ **Ahorro de tiempo**: No hay que escribir manualmente el problema
- ✅ **Mayor precisión**: La IA detecta y clasifica automáticamente
- ✅ **Priorización clara**: Clasificación por urgencia para atender primero lo crítico
- ✅ **Historial completo**: Todos los reportes quedan registrados
- ✅ **Accesible desde cualquier lugar**: Desplegado en la nube
- ✅ **Ubicación precisa**: Mapa interactivo para seleccionar el lugar exacto del problema

Objetivo General

El objetivo que se tiene para esta página es que la comunidad de CUCEI pueda reportar los problemas de mantenimiento de manera eficiente, utilizando inteligencia artificial para automatizar el análisis y clasificación de las incidencias.

Objetivos Específicos

Implementar un sistema de autenticación seguro (registro/login)	☑
Desarrollar un formulario de reporte con subida de imágenes	☑

Integrar Google Gemini AI para análisis automático de imágenes	☑
Configurar un mapa interactivo del CUCEI para seleccionar ubicaciones	☑
Crear un feed de reportes con funcionalidades sociales (likes, comentarios)	☑
Implementar sistema de comentarios con respuestas anidadas	☑
Permitir compartir reportes en redes sociales	☑
Almacenar imágenes de forma permanente en la nube	☑
Desplegar la aplicación en la nube (Render)	☑
Hacer la interfaz responsive para dispositivos móviles	☑
Optimizar la IA para ser más crítica en la evaluación de urgencia	☑
Implementar eliminación de reportes (solo para el dueño)	☑

Tecnologías utilizadas

Para este proyecto se utilizaron para las siguientes tecnologías

Backend

Tecnología	Versión	Propósito
Node.js	18.18.0	Entorno de ejecución JavaScript
Express.js	4.18.2	Framework web para Node.js
SQLite3	5.1.6	Base de datos ligera
Google Gemini AI	0.1.3	SDK para la API de Gemini
Cloudinary	1.41.0	Almacenamiento de imágenes en la nube
Multer	1.4.5	Middleware para subida de archivos
Bcryptjs	2.4.3	Encriptación de contraseñas

Frontend

Tecnología	Versión	Propósito
React	18.2.0	Biblioteca para UI
Vite	4.4.5	Build tool
React Bootstrap	2.10.0	Componentes UI
Axios	1.6.0	Peticiones HTTP
React Leaflet	4.2.1	Mapas interactivos

Arquitectura del proyecto

El flujo de datos sigue un modelo de cliente-servidor seguro:

1. **Frontend:** Captura la imagen y la ubicación del usuario.
2. **Servidor:** Procesa la imagen, consulta a la IA y sube el archivo a Cloudinary.
3. **Base de Datos:** Almacena la URL de la imagen y los datos analizados por la IA.

Estructura del proyecto

```
proyecto2/
├── backend/                # Servidor con Node.js
│   ├── server.js          # Código principal del backend
│   ├── package.json       # Dependencias del backend
│   └── database.db        # Base de datos SQLite
├── frontend/               # Aplicación React
│   ├── public/
│   │   └── images/
│   │       └── mapacucei.jpg   # Mapa del CUCEI
│   ├── src/
│   │   ├── components/   # Componentes de React
│   │   │   ├── login.jsx
│   │   │   ├── registro.jsx
│   │   │   ├── reporteform.jsx
│   │   │   ├── ReportesList.jsx
│   │   │   ├── MapaImagenSelector.jsx  # Mapa interactivo
│   │   │   ├── MenuLateral.jsx
│   │   │   └── Confetti.jsx
│   │   ├── services/
│   │   │   └── api.js
│   │   ├── App.jsx
│   │   ├── App.css
│   │   └── main.jsx
│   └── package.json
├── render.yaml            # Configuración para despliegue
└── README.md
```

Mapa Interactivo del CUCEI

Este proyecto incluye un mapa interactivo del centro universitario de ciencia exactos e ingeniería (CUCEI)

Descripción del mapa

Este mapa usa una imagen del mapa de cucei visto desde arriba sobre el cual se ha definido mas de 50 puntos de interés. Los usuarios pueden interactuar con el mapa de las siguientes maneras:

- **Selección por clic**: El usuario puede hacer clic en cualquier punto del mapa y el sistema detectará automáticamente la ubicación más cercana entre los módulos predefinidos. Si el clic está lejos de cualquier módulo, se guarda como una "ubicación personalizada" con las coordenadas exactas donde se hizo clic.
- **Selección por lista desplegable**: El usuario también puede elegir una ubicación de una lista que contiene todos los módulos disponibles, organizados por nombre y descripción.
- **Buscador de módulos**: El mapa incluye un campo de búsqueda que permite filtrar los módulos por nombre o descripción, facilitando la localización de ubicaciones específicas.
- **Marcadores visuales**: Cada módulo predefinido está representado por un punto de color azul con su nombre visible permanentemente. Al pasar el mouse sobre un punto, se muestra una descripción adicional del lugar.
- **Retroalimentación visual**: Cuando el usuario mueve el mouse sobre el mapa, aparece un punto amarillo que indica la posición actual del cursor. Al hacer clic para seleccionar una ubicación, aparece un punto rojo con una etiqueta que confirma la selección. Si el usuario selecciona un módulo predefinido, el punto cambia a color verde.
- **Leyenda de colores**: El mapa incluye una leyenda que explica el significado de cada color:
    - Puntos azules: Ubicaciones disponibles (módulos predefinidos)
    - Punto verde: Ubicación seleccionada (cuando se elige un módulo)

    - Punto amarillo: Posición actual del mouse
    - Punto rojo: Ubicación marcada por el usuario (ubicación personalizada)
- **Coordenadas relativas**: El sistema utiliza coordenadas en porcentaje (0% a 100%) en lugar de píxeles, lo que hace que el mapa sea completamente responsive y funcione correctamente en cualquier tamaño de pantalla, desde computadoras de escritorio hasta teléfonos móviles.
- **Campo de ubicación seleccionada**: Debajo del mapa, hay un campo de texto que muestra la ubicación que el usuario ha seleccionado, ya sea el nombre de un módulo o las coordenadas en porcentaje si se trata de una ubicación personalizada.

Configuración de la IA (Google Gemini)

¿Por qué se eligió Gemini?

La razón por la que se eligió Gemini fue porque era una IA que tiene una versión gratuita muy generosa (hasta 1,500 peticiones/día), además de ser más preciso ya que entiende los contextos complejos en imágenes.

Modelo usado

Se uso el modelo de Gemini 2.5-flash ya que su velocidad en respuesta es rápida, además estaba en la capa Free, es decir gratuito.

Configuración del Cliente

1. **Importación del SDK**: Se importa la librería oficial de Google
2. **Configuración de API key**: Se lee desde variables de entorno (nunca en código)
3. **Inicialización**: Se crea una instancia del cliente con la API key
4. **Selección del modelo**: Se elige gemini-2.5-flash por su equilibrio entre velocidad y calidad

El Prompt

El prompt es el conjunto de instrucciones que se le dan a la IA para que realice una tarea especifica. En este proyecto, el prompt está diseñado para que la IA sea critica y estricta al evaluar la urgencia.

A continuación, está el prompt completo de la página

const prompt = ` Eres un experto en seguridad y mantenimiento. Analiza esta imagen y responde SOLO con JSON.

REGLAS ESTRICTAS PARA LA URGENCIA:

- ALTA: Situación de PELIGRO INMEDIATO (fuego, agua淹, humo, cables expuestos,

  riesgo de caída, estructuras colapsando, personas en riesgo)

- ALTA: Daños ESTRUCTURALES GRAVES (paredes con grietas grandes, techos hundidos,

  columnas dañadas)

- MEDIA: Problemas que afectan el funcionamiento diario, pero SIN RIESGO DE VIDA

  (equipo roto, goteras menores, falta de suministros)

- BAJA: Problemas estéticos o menores sin impacto en seguridad

  (pintura descascarada, basura ligera, detalles estéticos)

REGLAS PARA CATEGORÍA:

- Infraestructura: Daños estructurales, instalaciones fijas

- Limpieza: Basura, suciedad, malos olores

- Seguridad: Riesgos para personas (solo si hay PELIGRO REAL)

- Tecnología: Equipos electrónicos dañados

- Servicios: Problemas de suministro (agua, luz, gas)

Responde con:

{"problema":"descripción clara y específica","categoria":"Infraestructura/Limpieza/Seguridad/Tecnología/Servicios","urgencia":"Baja/Media/Alta"}

NO USES "Alta" si no hay peligro real. Sé CRÍTICO. `;

¿Por qué este prompt está estructurado así?

Sección	Propósito
Rol asignado	"Eres un experto en seguridad y mantenimiento" → Le da contexto y autoridad a la IA
Reglas de urgencia	Define claramente qué significa cada nivel, evitando interpretaciones ambiguas
Reglas de categoría	Clasifica los tipos de problemas según su naturaleza
Formato de respuesta	JSON estructurado para facilitar el parseo en el backend
Instrucción final	"Sé CRÍTICO" → Refuerza la necesidad de ser estricto

Envío de la imagen a Gemini

```
// 1. Convertir la imagen a base64

const imageBuffer = req.file.buffer;

const base64Image = imageBuffer.toString("base64");


// 2. Enviar a Gemini con prompt e imagen

const result = await model.generateContent([

  prompt,
```

```
  { inlineData: { data: base64Image, mimeType: req.file.mimetype } }
]);

// 3. Obtener la respuesta
const responseText = result.response.text();
```

Explicación técnica:

Paso	Descripción
Buffer a base64	La imagen subida está en buffer (memoria). Se convierte a base64 para enviarla por HTTP
generateContent	Método que envía el prompt y la imagen a la API de Gemini
Respuesta asíncrona	La IA procesa la imagen y devuelve un texto (JSON en nuestro caso)

Flujo completo del análisis con la IA

1. **El usuario sube una imagen** a través del formulario de reporte en el frontend.
2. **El backend recibe la imagen** y la convierte a formato base64, que es un formato de texto que puede ser enviado a través de HTTP.
3. **Se envía la imagen a Gemini API** junto con el prompt que contiene todas las instrucciones sobre cómo debe analizarla y qué formato de respuesta debe usar.
4. **Gemini procesa la imagen**: identifica los objetos presentes, evalúa el nivel de peligro según las reglas establecidas, clasifica el problema en una de las cinco categorías disponibles y asigna un nivel de urgencia (Baja, Media o Alta).
5. **El backend recibe la respuesta** en formato JSON y la procesa para extraer los datos.

6. **Se aplica una validación adicional**: si Gemini asignó "Alta" pero la descripción del problema no contiene palabras clave como "peligro", "riesgo", "fuego", "grieta", "caída" o "expuestos", el sistema reduce automáticamente la urgencia a "Media" para evitar falsas alarmas.
7. **El reporte se guarda en la base de datos** con todos los datos analizados por la IA, incluyendo la descripción del problema, la categoría y la urgencia.
8. **La imagen se sube a Cloudinary** para almacenamiento permanente en la nube, lo que garantiza que no se pierda incluso si el servidor se reinicia.

Ejemplo de análisis

Tipo de imagen	Descripción del problema	Categoría	Urgencia
Pared con grieta grande	Grieta estructural en pared de concreto	Infraestructura	Alta
Cable eléctrico expuesto	Cables eléctricos expuestos en el suelo	Seguridad	Alta
Basura acumulada	Acumulación de basura en esquina	Limpieza	Media
Computadora apagada	Equipo de cómputo no enciende	Tecnología	Media
Pintura descascarada	Pintura deteriorada en pared	Infraestructura	Baja
Fuga de agua	Fuga de agua en tubería	Infraestructura	Alta
Lámpara fundida	Luminaria no funciona	Servicios	Media
Extintor vencido	Extintor con fecha vencida	Seguridad	Alta

Seguridad y Robustez

- **Autenticación:** Uso de bcryptjs para el hash de contraseñas, asegurando que los datos de los usuarios no sean legibles en la base de datos.
- **Gestión de Archivos:** Implementación de Multer para filtrar tipos de archivos y evitar la subida de scripts maliciosos.

- **Protección de API Keys:** Las credenciales de Gemini y Cloudinary se gestionan mediante variables de entorno (.env), nunca expuestas en el código fuente.
- **Filtro de "Falsos Positivos":** El sistema incluye una capa de lógica adicional que reduce la urgencia de "Alta" a "Media" si la descripción de la IA no detecta palabras de riesgo inminente.

Conclusión

Gracias a este proyecto podemos observar como la implementación de la IA puede optimizar drásticamente la gestión de infraestructura. Al automatizar la clasificación de urgencia y ubicación, se eliminan los errores humanos y se agiliza la atención de riesgos críticos.